Índice

1. Producto de un vectores de $\mathbb{R}^n$ por un escalar - Vídeo 04 **2**
1.1. Escena 1 - Producto de vectores de de $\mathbb{R}^n$ por escalares 2
1.2. Escena 2 - Interpretación geométrica del producto de números 6
1.3. Escena 3 - Interpretación geométrica del producto de vectores de de $\mathbb{R}^2$ 16
1.4. Escena 4 - Interpretación geométrica del producto de tres números (3D) 26
1.4.1. Escena 3D 26
1.4.2. Escena + Narración 29
1.5. Escena 5 - Resumen 31

2. Trozos comunes de código **32**
2.1. Carga de la librería Manim y NacAL 32

3. Rodando: 1,2,3... ¡acción! **36**

Lección 1 del curso - Vídeo 4

Marcos Bujosa

27 de enero de 2026

1. Producto de un vectores de $\mathbb{R}^n$ por un escalar - Vídeo 04

1.1. Escena 1 - Producto de vectores de de $\mathbb{R}^n$ por escalares

```
<<Carga de la librería Manim y NacAL con AzureService>>
#<<Carga de la librería Manim y NacAL>>
<<Definición de mi escena con voz>>
<<Definición de VectorR2>>
<<Definición de VectorR3>>
<<copyright>>

class L01_V04_E01_ProductoPorEscalares(MiEscenaConVoz):
    def construct(self):
        self.set_speech_service( AzureService(voice="es-ES-AlvaroNeural" ) )
        #self.set_speech_service(GTTSService(lang="es", tld="com"))
        myTemplate = TexTemplate()
        myTemplate.add_to_preamble(r"""\usepackage{nacal} """)

        self.creditos()

        # Portada
        titulo = Title(_("Producto de vectores de ") + r"\R[n]" + _("por escalares"),
                    tex_template = myTemplate,
                    font_size=70).set_color(BLUE)
        self.play(Write(titulo))
        self.pausa_media()
        self.play(FadeOut(titulo))

        operacionProducto = Tex(_("Producto por escalares"),
                        tex_template = myTemplate, font_size=70
                        ).to_edge(UP).set_color(BLUE)

        operacionDescripcion = Tex(_("El producto se define componente a componente."),
                        tex_template = myTemplate,
                        ).move_to([0,2.5,0]).to_edge(LEFT)
        # Ejemplos
        EjR3 = Tex(r"\textbf{" + _("Ejemplo en ") + r"\R[3]:}",
                 tex_template = myTemplate,
                 font_size=50).set_color(GREEN).next_to(operacionDescripcion, DOWN, aligned_edge=LEFT)

        EjR4 = Tex(r"\textbf{" + _("Ejemplo en ") + r"\R[4]:}",
                 tex_template = myTemplate,
                 font_size=50).set_color(GREEN).next_to(operacionDescripcion, DOWN, aligned_edge=LEFT)

        a    = 3
        b    = nc.Vector( [0, 1, 2])
        s1 = MathTex(a,                tex_template = myTemplate,)
        por= MathTex(r"\cdot",         tex_template = myTemplate,)
        s3 = MathTex(b.latex(),        tex_template = myTemplate,)
        igual = MathTex(r"=",          tex_template = myTemplate,)
        s5 = MathTex((a*b).latex(),    tex_template = myTemplate,)
        grp1 = VGroup(s1,por,s3,igual,s5).arrange(RIGHT)
```

```
with self.voiceover(text=_("""Además de sumar vectores,""")) as tracker:
    self.pausa_corta(tracker.duration)

self.add(operacionProducto)
self.add(operacionDescripcion)

with self.voiceover(text=_("""también podemos reescalarlos, es decir, podemos multiplicarlos
por algún escalar para obtener un múltiplo del vector original.""")) as tracker:
    self.add(EjR3)
    self.play(FadeIn(grp1[2]), run_time=tracker.duration/4)
    self.play(FadeIn(grp1[:2]), run_time=tracker.duration*3/4)

with self.voiceover(text=_("""El resultado es otro vector que se define componente a
componente.""")) as tracker:
    self.pausa_media()
    self.add(grp1[3])
    self.add(grp1[4][0][:2])
    self.add(grp1[4][0][-2:])

with self.voiceover(text=_("""La primera componente del múltiplo es el producto del escalar
por la primera componente del vector.""")) as tracker:
    self.play(FadeIn(grp1[4][0][2]), run_time=tracker.duration/3)
    self.play(Circumscribe(grp1[0]),
              Circumscribe(grp1[2][0][2]),
              Indicate(grp1[4][0][2]),
              run_time=tracker.duration*2/3)
    self.pausa_corta()

with self.voiceover(text=_("""De manera similar se calculan tanto la segunda
componente, como las restantes componentes del múltiplo del vector.""")) as tracker:
    self.play(FadeIn(grp1[4][0][3]), run_time=tracker.duration/3)
    self.play(Circumscribe(grp1[0]),
              Circumscribe(grp1[2][0][3]),
              Indicate(grp1[4][0][3]),
              run_time=tracker.duration/3)
    self.play(FadeIn(grp1[4][0][4]),
              run_time=tracker.duration/6)
    self.play(Circumscribe(grp1[0]),
              Circumscribe(grp1[2][0][4]),
              Indicate(grp1[4][0][4]),
              run_time=tracker.duration/3 )
    self.pausa_muy_corta()
    self.play(FadeOut(grp1))

escalar = sp.symbols('\lambda')
vga = MathTex(escalar, tex_template = myTemplate)

v_generico_b  = nc.Vector(sp.symbols('b:5')[1:])
vgb = MathTex(v_generico_b.latex(), tex_template = myTemplate)

vgab = MathTex((escalar * v_generico_b).latex(), tex_template = myTemplate)

grp2 = VGroup(vga,por,vgb,igual,vgab).arrange(RIGHT)
with self.voiceover(text=_("""Así, la siguiente expresión describe
el producto de un vector de R 4 por un escalar.""")) as tracker:
    self.play(FadeTransform(EjR3,EjR4))
    self.play(FadeIn(grp2))
    self.pausa()

with self.voiceover(text=_("""Lo habitual es simplificar la notación,
omitiendo el punto que denota la operación producto""")) as tracker:
    self.pausa(tracker.duration/3)
    self.play(FadeOut(grp2[1]),
              grp2[0].animate.move_to(grp2[1]),
              run_time=2*tracker.duration/3)
    self.pausa_muy_corta()
```

```
with self.voiceover(text=_("""Como en el caso de la suma,""")) as tracker:
    self.pausa(tracker.duration)

with self.voiceover(text=_("""para definir la operación en R n necesitamos una
estrategia que no requiera escribir la lista de componentes. Recuerde que n puede
ser muy grande.""")) as tracker:
    self.wait(tracker.duration/6)
    self.play(Indicate(vga[0][4:-4]),
              Indicate(vgb[0][4:-4]),
              Indicate(vgab[0][4:-4]),
              run_time=2*tracker.duration/3)
    self.wait(1.5*tracker.duration/6)

Defn = Tex(r"\textbf{" + _("Definición") + ":}",
         tex_template = myTemplate,
         font_size=50).set_color(RED).next_to(operacionDescripcion,
                                              DOWN,
                                              aligned_edge=LEFT)

with self.voiceover(text=_("""Una alternativa es definir el producto usando el operador
selector. Con ella podemos expresar""")) as tracker:
    self.play(FadeOut(grp2),
              FadeOut(EjR4),
              run_time=tracker.duration/3)
    self.add(Defn)

cvab = MathTex(r"\elemRp{\lambda\Vect{a}}{i}", tex_template = myTemplate)
cvb  = MathTex(r"\lambda\eleVRpE{a}{i}", tex_template = myTemplate)
eq_suma = VGroup(cvab,igual,cvb).arrange(RIGHT).scale(1.5)

donde = Tex(_("donde"))
indices = MathTex(r"i=1:n", tex_template = myTemplate)
pc_indices = VGroup(donde,indices).arrange(RIGHT, buff=1)
grp3 = VGroup(eq_suma, pc_indices).arrange(RIGHT, buff=1)

with self.voiceover(text=_("""que la i-ésima componente del
múltiplo del vector es igual""")) as tracker:

    self.play(FadeIn(grp3[0][0],
                     scale=1.5,
                     rate_func=rate_functions.exponential_decay),
              run_time=tracker.duration-1.5)
    self.play(FadeIn(grp3[0][1],
                     scale=1.5,
                     rate_func=rate_functions.exponential_decay))

with self.voiceover(text=_("""al escalar por la i-ésima
componente del vector.""")) as tracker:

    self.play(FadeIn(grp3[0][2:],
                     scale=0.5,
                     rate_func=rate_functions.exponential_decay),
              run_time=tracker.duration+0.2)

with self.voiceover(text=_(r"""(donde el índice recorre los números naturales entre uno
y n)""")) as tracker:
    self.play(FadeIn(grp3[1]))
    self.pausa_corta()

with self.voiceover(text=_(r"""Esta definición abstracta  arroja una segunda regla de
cálculo simbólico:""")) as tracker:
    self.pausa(tracker.duration*2/3)
    self.play(Indicate(eq_suma[0][0][0]),
              Indicate(eq_suma[0][0][3]),
              Indicate(eq_suma[2][0][1]),
              Indicate(eq_suma[2][0][5]),
              run_time=tracker.duration/3)
    self.pausa_muy_corta()
```

```
with self.voiceover(text=_(r"""que la i-ésima componente del producto se puede sustituir por
el producto del escalar por la i-ésima componente.""")) as tracker:
    source0 = MathTex(r"\elemRp{\lambda\Vect{a}}{i}",
                      tex_template = myTemplate).next_to(grp3, DOWN, buff=1.2).scale(2)[0]
    target0 = MathTex(r"\lambda\eleVRpE{a}{i}",
                      tex_template = myTemplate).next_to(grp3, DOWN, buff=1.2).scale(2)[0]
    source1 = target0.copy()
    target1 = source0.copy()

    source2 = source0.copy()
    target2 = target0.copy()

    VG0 = VGroup(source0,target0)
    self.add(source0)
    transform_index0 = [[0,1,2,3,4,5],
                        [1,0,2,5,3,4]]
    self.play(
        *[
            ReplacementTransform(source0[i],target0[j], rate_func=rate_functions.smooth)
            for i,j in zip(*transform_index0)
        ],
        run_time=tracker.duration)

with self.voiceover(text=_("""Y que el escalar por la i-ésima componente se puede
sustituir por la i-ésima componente del producto.""")) as tracker:
    self.play(ReplacementTransform(target0,source1))

    VG1 = VGroup(source1,target1)
    transform_index1 = [[0,1,2,3,4,5],
                        [1,0,2,4,5,3]]
    self.play(
        *[
            ReplacementTransform(source1[i],target1[j], rate_func=rate_functions.smooth)
            for i,j in zip(*transform_index1)
        ],
        run_time=tracker.duration)

with self.voiceover(text=_("""Es decir, que los paréntesis pueden desplazarse para dejar
fuera al escalar y dentro el operador selector, y viceversa.""")) as tracker:
    self.play(ReplacementTransform(target1,source2))
    VG2 = VGroup(source2,target2)
    self.play(
        *[
            ReplacementTransform(source2[i],target2[j], rate_func=rate_functions.there_and_back_with_pause)
            for i,j in zip(*transform_index0)
        ],
        run_time=tracker.duration)

with self.voiceover(text=_("""Esta regla se denomina propiedad asociativa del operador selector
respecto del producto.""")) as tracker:
    self.remove(target1,source2,target2)
    self.play(Indicate(eq_suma[0][0][0]),
              Indicate(eq_suma[0][0][3]),
              Indicate(eq_suma[2][0][1]),
              Indicate(eq_suma[2][0][5]),
              run_time=tracker.duration)
    self.play(FadeOut(grp3), run_time=0.3)
    self.pausa_muy_corta()

with self.voiceover(text=_("""Por tanto, los paréntesis no son
necesarios, pues la expresión sin ellos tiene pleno sentido;
tal y como ocurre con el producto entre números.""")) as tracker:

    target3 = MathTex(r"\lambda\eleVR{a}{i}",
                      tex_template = myTemplate).next_to(grp3, DOWN, buff=0).scale(1.5)[0]
    grp4 = VGroup(cvab,igual,cvb,igual.copy(),target3).arrange(RIGHT, buff=1)
```

```
        producto = MathTex(r"(ab)c\;=\;a(bc)\;=\;abc",
                           tex_template = myTemplate).next_to(grp4, DOWN, buff=1.2).scale(1.5)[0]

        self.play(FadeIn(grp4),
                  run_time=tracker.duration/3)

        self.play(Indicate(target3), run_time=tracker.duration/3)

        self.play(FadeIn(producto))

        self.pausa(tracker.duration/3)
        self.pausa_muy_corta(1.5)
```

1.2. Escena 2 - Interpretación geométrica del producto de números

---

```
class L01_V04_E02_InterpretacionGeometricaDelProductoDeDosNumeros(MiEscenaConVoz):
    def construct(self):
        self.set_speech_service( AzureService(voice="es-ES-AlvaroNeural" ) )
        #self.set_speech_service(GTTSService(lang="es", tld="com"))
        myTemplate = TexTemplate()
        myTemplate.add_to_preamble(r"""\usepackage{nacal} """)

        cabecera = Tex(_("Interpretación geométrica del producto"),
                           tex_template = myTemplate,
                           font_size=70).to_edge(UP).set_color(BLUE)

        cabecera2 = Tex(_("Interpretación geométrica del producto de números"),
                        tex_template = myTemplate,
                        font_size=70).to_edge(UP).set_color(BLUE).scale(0.6).shift(0.1*UP)

        def coordenada_x(x,ax, label='', pos=0):
            if x==int(x):
                xlabel = (ax[0][2][int(x)-1]).copy()
            else:
                label = label if label else f"{x}"
                xlabel = Tex(label, font_size=30)
                pos_label = UP if pos else DOWN
                xlabel.next_to(ax.c2p(x,0), pos_label, buff=0.6)
            return xlabel

        def coordenada_y(y,ax, label='', pos=0):
            if y==int(y):
                ylabel = (ax[1][2][int(y)-1]).copy()
            else:
                label = label if label else f"{y}"
                ylabel = Tex(label, font_size=30)
                pos_label = RIGHT if pos else LEFT
                ylabel.next_to(ax.c2p(0,y), pos_label, buff=0.5)
            return ylabel

        coordenadas_xy = lambda x,y,ax,labelx='',labely='',posx=0,posy=0: (coordenada_x(x,ax,labelx,posx),
                                                                           coordenada_y(y,ax,labely,posy))

        def retículaProducto(a, b: int, ax: Axes) -> VGroup:
            def cuadricula(a: int, b: int, ax: Axes):
                base = VGroup(*[Square().scale(0.5) for s in range(0,int(a*b))])
                base.arrange_in_grid(rows=b,
                                     flow_order="ru",
                                     buff=0).next_to(ax.coords_to_point(0, 0),
                                                     UR,
                                                     buff=0)
                return Group(*[base[c[0]:c[1]] for c in [(a*i,a*(i+1)) for i in range(b)]])

            def rectangulo(a, b: int, ax: Axes):
                base  = VGroup(*[Rectangle(height=2.0, width=2.0*a).scale(0.5) for s in range(0,b)])
```

```
        posicion = UR if a>0 else UL
        tabla = base.arrange_in_grid(rows=b,
                                     flow_order="ru",
                                     buff=0).next_to(ax.coords_to_point(0, 0),
                                                     posicion,
                                                     buff=0)
        return tabla
    return cuadricula(a, b, ax) if isinstance(a, int) else rectangulo(a, b, ax)

def productoEnteros(a,b,ax,tabla,duración=2):
    print(len(tabla))
    self.play([fila.animate.shift((len(tabla)-1-i)*a*RIGHT) for i,fila in enumerate(tabla)], run_time=duración/2)
    self.play([fila.animate.shift(i*DOWN) for i,fila in enumerate(tabla)], run_time=duración/2)

def producto(a,b,ax,tabla,duración=3):
    self.play([fila.animate.shift((len(tabla)-1-i)*a*RIGHT) for i,fila in enumerate(tabla)], run_time=duración/3)
    self.play([fila.animate.shift(i*DOWN) for i,fila in enumerate(tabla)], run_time=duración/3)
    self.play([fila.animate.shift(i*UP) for i,fila in enumerate(tabla)], run_time=duración/3)

def Arco(ax, b):
    unidad = 1
    arco = Arc(start_angle=PI/2,  # Ángulo de inicio (90 grados en radianes)
               angle=-PI/2,       # Ángulo de arco (-90 grados en radianes)
               radius=b*unidad,     # Radio del arco
               arc_center=ax.c2p(0,0),
               stroke_width=0.8)
    return arco

def Rectángulo(a, b, ax: Axes):
        posición = UR if a>0 else UL
        rectángulo = Rectangle(height=1.0,
                               width=1.0*a).next_to(ax.coords_to_point(0, 0),
                                                    posición,
                                                    buff=0)
        return rectángulo

def multiplica(a, b, ax: Axes):
    """
    Desplaza horizontalmente una recta que pasa por (a, 0) y (0, 1)
    hasta que corte el eje vertical en el punto (0, b)
    """
    # Diagonal del rectángulo
    punto_inferior = ax.coords_to_point(a, 0)
    punto_superior = ax. coords_to_point(0, 1)
    diagonal = Line(punto_inferior, punto_superior, color=BLUE)
    # Extender la diagonal.
    # Con la ecuación: y = 1 - x/a, podemos calcular puntos más alejados
    if b>0:
        y_arriba = 1+b # Punto lejano a la izquierda
        x_arriba = -a*(y_arriba-1)
        y_abajo = -1  # Punto lejano a la derecha
        x_abajo = 2*a
    else:
        y_arriba = 2 # Punto lejano a la izquierda
        x_arriba = -a
        y_abajo = b-1  # Punto lejano a la derecha
        x_abajo = -a*(y_abajo-1)
        # Diagonal extendida para cortar los ejes tras el desplazamiento
    diagonal_extendida = Line(
        ax.coords_to_point(x_arriba, y_arriba),
        ax.coords_to_point(x_abajo, y_abajo),
        color=RED                )
    desplazamiento_x = a * (b - 1)
    desplazamiento = ax.coords_to_point(desplazamiento_x, 0) - ax.coords_to_point(0, 0)
    diagonal_extendida.shift(desplazamiento)

    return diagonal, diagonal_extendida
```

```
## arranque del vídeo
self.creditos(1)

ax = Axes(
    x_length=12,
    y_length=6,
    x_range=[0, 12, 1],
    y_range=[0, 6, 1],
    tips=False,
    axis_config={"include_numbers": True}
)

resumen1 = Tex(_(r"""Para interpretar geométricamente el producto por escalares en \R[2],"""),
               tex_template = myTemplate).scale_to_fit_width(config.frame_width*.9)

resumen2 = Tex(_(r"""antes interpretaremos geométricamente el producto de dos números."""),
               tex_template = myTemplate).next_to(resumen1, DOWN).scale_to_fit_width(config.frame_width*.9)

with self.voiceover(text=_("""Antes de ver la interpretación
geométrica del reescalado de vectores en R2,""")) as tracker:

    self.add(cabecera)
    self.play(Write(resumen1), run_time=tracker.duration+0.2)

with self.voiceover(text=_("""debemos dar una interpretación
geométrica al producto entre números reales.""")) as tracker:

    self.play(Write(resumen2), run_time=tracker.duration)
    self.play(FadeOut(resumen1),
              FadeOut(resumen2))

with self.voiceover(text=_("""Cuando los números son naturales
es habitual asociar sus magnitudes con longitudes, y los
productos con superficies""")) as tracker:

    self.play(FadeIn(ax),
              FadeTransform(cabecera, cabecera2),
              run_time=tracker.duration/3)

with self.voiceover(text=_("""Así, el producto de uno por uno
se asocia con un cuadrado de lado 1""")) as tracker:

    a=1
    b=1
    tabla = retículaProducto(a,b,ax)
    self.pausa(tracker.duration/6)
    xlabel,ylabel=coordenadas_xy(a,b,ax)
    self.play(Indicate(xlabel,scale_factor=2.5))
    self.play(Indicate(ylabel,scale_factor=2.5))
    self.remove(xlabel,
                ylabel)
    [self.play(FadeIn(base), run_time=.5) for base in tabla]
    self.play(Indicate(tabla), run_time=tracker.duration/3)

with self.voiceover(text=_("""Evidentemente el área del
cuadrado coincide con la longitud de la base.""")) as tracker:
    self.pausa(tracker.duration/2)
    self.play(tabla.animate.fade(0.8), run_time=tracker.duration/2)


with self.voiceover(text=_("""Podemos asociar el producto tres
por uno con un rectángulo de base tres y altura uno.""")) as tracker:

    a=3.
    b=1
    tabla2 = retículaProducto(a,b,ax)
    xlabel,ylabel=coordenadas_xy(a,b,ax)
    self.play(Indicate(xlabel,scale_factor=2.5))
```

```
        self.play(Indicate(ylabel,scale_factor=2.5))
        self.remove(xlabel,
                    ylabel)
        [self.play(FadeIn(base), run_time=.5) for base in tabla2]
        self.play(Indicate(tabla2), run_time=tracker.duration/3)
        self.pausa(tracker.duration/4)

    with self.voiceover(text=_("""Y de nuevo el área coincide con
    la longitud de la base, pues la altura es uno.""")) as tracker:
        self.play(FadeOut(tabla), run_time=tracker.duration/2)
        self.play(tabla2.animate.set_opacity(0.15), run_time=tracker.duration/2)

    with self.voiceover(text=_("""Multiplicar uno por tres
    corresponde a apilar tres cuadrados de base uno. Si los
    alineamos en una fila, podemos comprobar que el
    área total (así como la longitud) es igual a la del rectángulo
    tres por uno.""")) as tracker:

        a=1
        b=3
        tabla = retículaProducto(a,b,ax)
        xlabel,ylabel=coordenadas_xy(a,b,ax)
        self.play(Indicate(xlabel,scale_factor=2.5))
        self.play(Indicate(ylabel,scale_factor=2.5))
        self.remove(xlabel,
                    ylabel)
        self.play([FadeIn(base) for base in tabla],
                  run_time=tracker.duration/3)
        productoEnteros(a,b,ax,tabla)
        self.pausa(tracker.duration/3)
        self.play(FadeOut(tabla),
                  FadeOut(tabla2))

    with self.voiceover(text=_("""Para multiplicar tres por cuatro
    apilamos cuatro rectángulos de base tres y altura uno. De
    nuevo, si los alineamos en una fila, su longitud
    total es el valor del producto.""")) as tracker:

        a=3.
        b=4
        tabla = retículaProducto(a,b,ax)
        xlabel,ylabel=coordenadas_xy(a,b,ax)
        self.play(Indicate(xlabel,scale_factor=2.5))
        self.play(Indicate(ylabel,scale_factor=2.5))
        self.remove(xlabel,
                    ylabel)
        self.play([FadeIn(base) for base in tabla], run_time=tracker.duration/3)
        productoEnteros(a,b,ax,tabla,duración=tracker.duration/3)
        self.pausa_muy_corta()
        self.play(FadeOut(tabla))

    with self.voiceover(text=_("""Esta forma de multiplicar se
    puede generalizar a rectángulos cuya base tenga cualquier
    longitud; siempre y cuando usemos un número "natural" de
    rectángulos..""")) as tracker:

        a=2/5
        b=6
        tabla = retículaProducto(a,b,ax)
        xlabel,ylabel=coordenadas_xy(a,b,ax)
        self.play(Indicate(xlabel,scale_factor=2.5))
        self.play(Indicate(ylabel,scale_factor=2.5))
        self.remove(xlabel,
                    ylabel)
        self.play([FadeIn(base) for base in tabla], run_time=tracker.duration/4)
        productoEnteros(a,b,ax,tabla,duración=tracker.duration/3)
        self.play(FadeOut(tabla),
                  run_time=tracker.duration/4)
```

```
with self.voiceover(text=_("""Por ejemplo, podemos multiplicar
raíz de dos por cinco.""")) as tracker:
    a=np.sqrt(2)
    a_label=r"$\sqrt{2}$"
    b=5
    tabla = retículaProducto(a,b,ax)
    self.pausa(tracker.duration/3)
    xlabel,ylabel=coordenadas_xy(a, b, ax, labelx=a_label)
    self.play(Indicate(xlabel,scale_factor=2.5))
    self.play(Indicate(ylabel,scale_factor=2.5))
    self.remove(ylabel)
    self.play([FadeIn(base) for base in tabla], run_time=tracker.duration/3)
    productoEnteros(a,b,ax,tabla)

with self.voiceover(text=_("""El resultado es algo mayor que
siete.""")) as tracker:

    self.pausa(tracker.duration)

with self.voiceover(text=_("""Pero ¿cómo podemos multiplicar
dos numeros reales? Necesitamos una nueva interpretación del
producto que no requiera una cantidad "entera" de
rectángulos.""")) as tracker:

    self.pausa(tracker.duration)

with self.voiceover(text=_("""Fijémonos en lo que pasa si nos
quedamos a mitad de camino entre la pila inicial y la fila
final.""")) as tracker:

    self.play([fila.animate.shift(i*UP) for i,fila in enumerate(tabla)],
              run_time=tracker.duration)

with self.voiceover(text=_("""En esta posición los rectángulos
también están alineados.""")) as tracker:

    recta =  Line(start = tabla.get_corner(DR),
                  end   = tabla.get_corner(UL),
                  color = RED_B)
    self.play(FadeIn(recta))

with self.voiceover(text=_("""La recta que atraviesa las
 diagonales, corta el eje de ordenadas en el valor por el que
 hemos multiplicado la base; y corta el eje de abcisas en el
 valor del resultado.""")) as tracker:

    xlabel5=coordenada_x(a*5, ax, label=r"$5\sqrt{2}$")
    self.pausa(tracker.duration/4)
    self.play(Indicate(ylabel, scale_factor=2.5), run_time=tracker.duration/3)
    self.play(Indicate(xlabel5, scale_factor=2.5), run_time=tracker.duration/3 )

with self.voiceover(text=_("""Lo más interesante es que esto es cierto con cualquier traslación de la recta.""")) as tra
    rectas = VGroup(*[Line(start = tabla.get_corner(UL),
                           end   = fila.get_corner(DR),
                           color = RED_B) for i, fila in enumerate(tabla)])
    xlabel2=coordenada_x(a*2, ax, label=r"$2\sqrt{2}$")
    xlabel3=coordenada_x(a*3, ax, label=r"$3\sqrt{2}$")
    xlabel4=coordenada_x(a*4, ax, label=r"$4\sqrt{2}$")
    xlabels=VGroup(reversed([xlabel,xlabel2,xlabel3,xlabel4,xlabel5]))

    for i, fila in reversed(list(enumerate(tabla))):
        self.play(rectas[i].animate.shift(i * DOWN),
                  fila.animate.shift(i*DOWN),
                  Indicate(xlabels[i], scale_factor=2.5),
                  Indicate(ax[1][2][len(tabla)-1-i], scale_factor=2.5),
                  run_time=tracker.duration/4)
    self.pausa()
```

```
        self.play(FadeOut(tabla,
                          recta,
                          rectas,
                          xlabel,
                          xlabels))

    with self.voiceover(text=_("""Para verlo, volvamos primero a
    un rectángulo cuya base tiene longitud entera.""")) as tracker:
        self.wait(tracker.duration)

    with self.voiceover(text=_("""tres por cuatro es doce, y se
    puede calcular trazando una recta paralela a la diagonal que
    corte el eje vertical a altura 4. Dicha recta cortará el eje
    horizontal en 12.""")) as tracker:

        a=3.
        b=4
        rectángulo = Rectángulo(a, b, ax)
        diagonal, diagonal_extendida = multiplica(a, b, ax)
        self.play(Create(rectángulo))
        self.play(Indicate(diagonal))
        self.play(ReplacementTransform(diagonal.copy(),diagonal_extendida))
        self.pausa()

    with self.voiceover(text=_("""Pero también se puede ver que 3 por 5 tercios es 5.""")) as tracker:

        a=3.
        b=5/3
        b_label=r"$\frac{5}{3}$"
        ylabelF=coordenada_y(b, ax, label=b_label)
        kk, diagonal_extendida2 = multiplica(a, b, ax)
        self.add(ylabelF)
        self.play(ReplacementTransform(diagonal.copy(),diagonal_extendida2))
        self.pausa()

    with self.voiceover(text=_("""O que tres por pi es algo menos de 9 y medio.""")) as tracker:
        a=3.
        b=np.pi
        b_label=r"$\pi$"
        ylabelpi=coordenada_y(b, ax, label=b_label)
        kk, diagonal_extendida3 = multiplica(a, b, ax)
        self.add(ylabelpi)
        self.play(ReplacementTransform(diagonal.copy(),diagonal_extendida3))
        self.pausa_larga()
        self.play(FadeOut(ylabelF,
                          ylabelpi,
                          diagonal_extendida,
                          diagonal_extendida2,
                          diagonal_extendida3,
                          rectángulo,
                          diagonal))

    with self.voiceover(text=_("""De este modo podemos multiplicar
    dos numeros reales cualesquiera. Por ejemplo, dos raíz de dos
    por tres raíz de dos.""")) as tracker:

        a=2*np.sqrt(2)
        a_label=r"$2\sqrt{2}$"
        b=3*np.sqrt(2)
        b_label=r"$3\sqrt{2}$"
        xlabel,ylabel=coordenadas_xy(a, b, ax, labelx=a_label, labely=b_label)
        rectángulo = Rectángulo(a, b, ax)

        self.pausa(tracker.duration/2)
        self.play(Indicate(xlabel,scale_factor=2.5),
                  FadeIn(rectángulo))
        self.play(Indicate(ylabel,scale_factor=2.5))
```

```
with self.voiceover(text=_("""El resultado es exactamente 12.""")) as tracker:
    diagonal, diagonal_extendida = multiplica(a, b, ax)
    self.play(Create(diagonal))
    self.play(ReplacementTransform(diagonal.copy(),diagonal_extendida))
    self.wait(tracker.duration)

    self.play(*[FadeOut(mob)for mob in self.mobjects] )  # borrado de todo


ax = Axes(
    x_length=12,
    y_length=6,
    x_range=[-6, 6, 1],
    y_range=[-3, 3, 1],
    tips=False,
    x_axis_config={"include_numbers": True}
)
self.add(ax)

a=-2.5
a_label=r"$\alpha$"
b=1
b=-1.5
b_label=r"$\beta$"

xlabel, ylabel = coordenadas_xy(a, b, ax, labelx=a_label, labely=b_label)
xlabel.scale(1.5)
ylabel.scale(1.5)

c=1.5
c_label=r"$\gamma$"
__, ylabel2 = coordenadas_xy(a, c, ax, labelx=a_label, labely=c_label)
ylabel2.scale(1.5)

with self.voiceover(text=_("""Así pues, para multiplicar
gráficamente un número alfa por un número beta:""")) as tracker:

    self.pausa(tracker.duration/3)
    self.play(Indicate(xlabel,scale_factor=1.5),
              run_time=tracker.duration/3)
    self.play(Indicate(ylabel,scale_factor=1.5),
              run_time=tracker.duration/3)

with self.voiceover(text=_("""Pintamos un rectángulo cuya base
va desde cero hasta alfa, y con altura uno.""")) as tracker:

    rectángulo = Rectángulo(a, b, ax)
    diagonal, diagonal_extendida = multiplica(a, b, ax)
    self.play(Indicate(rectángulo))

with self.voiceover(text=_("""y la diagonal que pasa por los
puntos alfa-cero y cero-uno.""")) as tracker:
    puntoA = MathTex(r"(\alpha, 0)",
                     tex_template = myTemplate)
    puntoB = MathTex(r"(0, 1)",
                     tex_template = myTemplate)
    segmento = VGroup(puntoA,
                      puntoB).arrange(RIGHT,
                                      buff=1.2).move_to([0,2.5,0]).to_edge(LEFT)
    self.play(Create(diagonal))
    self.play(Indicate(puntoA))
    self.play(Indicate(puntoB))


with self.voiceover(text=_("""Extendemos la diagonal.""")) as tracker:
    diagonal_alargada =diagonal.copy().scale(4)
    self.play(FadeIn(diagonal_alargada),
              FadeOut(rectángulo))
```

```
with self.voiceover(text=_("""y la trasladamos hasta que interseque con el eje vertical a altura beta.""")) as tracker:
    self.play(FadeTransform(diagonal_alargada,
                            diagonal_extendida),
              run_time=tracker.duration/2)
    self.play(Indicate(ylabel),
              scale_factor=2.5,
              run_time=tracker.duration/2)

with self.voiceover(text=_("""El punto de corte con el eje horizontal es el valor del producto alpha por beta""")) as tr

    prod_label = Tex(r"$\alpha\beta$", font_size=30).scale(1.5)
    prod_label.next_to(ax.c2p(a*b,0), UP, buff=0.3)

    self.wait(tracker.duration/2)
    self.play(Indicate(prod_label,
                       scale_factor=2.5),
              run_time=tracker.duration/2)

with self.voiceover(text=_("""Esta representación respeta los
signos del producto:""")) as tracker:

    self.pausa(tracker.duration)

with self.voiceover(text=_("""menos por menos es más. Por eso
alfa por beta es positivo en la figura.""")) as tracker:

    self.play(Indicate(xlabel,scale_factor=2),
              Indicate(ylabel,scale_factor=2),
              Indicate(prod_label,scale_factor=2),
              run_time=tracker.duration*2/3)

with self.voiceover(text=_("""Sin embargo, menos por más es
menos. Así que alfa por gamma tiene signo negativo.""")) as tracker:

    __, diagonal_extendida2 = multiplica(a, c, ax)
    prod_label2 = Tex(r"$\alpha\gamma$", font_size=30).scale(1.5)
    prod_label2.next_to(ax.c2p(a*c,0), UP, buff=0.3)

    self.play(Indicate(ylabel2, scale_factor=1.5),
              Indicate(xlabel,  scale_factor=1.5),
              run_time=tracker.duration/3)

    self.play(FadeTransform(diagonal.copy(),
                            diagonal_extendida2),
              run_time=tracker.duration/3),

    self.play(Indicate(prod_label2,
                       scale_factor=2.5),
              run_time=tracker.duration/3)
    self.wait()
    self.play(FadeOut(diagonal_extendida,
                      diagonal_extendida2,
                      prod_label,
                      prod_label2,
                      xlabel,
                      ylabel,
                      ylabel2,
                      puntoA,
                      puntoB,))



cerolabel = Tex('$0$', font_size=32, color=YELLOW).scale(1.3)
cerolabel.next_to(ax.c2p(0,0), DOWN)

with self.voiceover(text=_("""Nótese que si alfa es cero, la
diagonal se superpone sobre el eje vertical.""")) as tracker:
```

```
    punto_inferior = ax.coords_to_point(0, 0)
    punto_superior = ax. coords_to_point(0, 1)
    diagonal_vertical = Line(punto_inferior, punto_superior, color=BLUE)
    punto_inferiorMin = ax.coords_to_point(0, -4)
    punto_superiorMax = ax. coords_to_point(0, 4)
    diagonal_vertical_extendida = Line(punto_inferiorMin, punto_superiorMax, color=BLUE)

    self.play(ReplacementTransform(diagonal,diagonal_vertical),
              run_time=tracker.duration/2)

with self.voiceover(text=_("""Por lo que todos los puntos de
su extensión pertenecen al eje vertical, que corta al
horizontal en cero. Lo que muestra que todo múltiplo de cero
es cero.""")) as tracker:

    self.play(ReplacementTransform(diagonal_vertical,
                                   diagonal_vertical_extendida,
                                   rate_func=rate_functions.slow_into),
              run_time=tracker.duration/2)

    self.play(Indicate(cerolabel),
              run_time=tracker.duration/4)
    self.wait(tracker.duration/4)

    self.play(*[FadeOut(mob)for mob in self.mobjects] )  # borrado de todo

with self.voiceover(text=_("""Para finalizar, el producto de
dos números puede visualizarse gráficamente en cualquier
eje. Por ejemplo en el vertical.""")) as tracker:

    ax = Axes(
        x_length=12,
        y_length=6,
        x_range=[0, 12, 1],
        y_range=[0, 6, 1],
        tips=False,
        axis_config={"include_numbers": True}
    )

    self.play(FadeIn(ax),
              run_time=tracker.duration/2)


def multiplicaEjeVertical(a, b, ax: Axes):
    """
    Desplaza horizontalmente una recta que pasa por (a, 0) y (0, 1)
    hasta que corte el eje vertical en el punto (0, b)
    """
    # Diagonal del rectángulo
    punto_inferior = ax.coords_to_point(1, 0)
    punto_superior = ax. coords_to_point(0, a)
    diagonal = Line(punto_inferior, punto_superior, color=BLUE)
    # Extender la diagonal.
    # Con la ecuación: y = a - b*x, podemos calcular puntos más alejados
    #if b>0:
    y_arriba = a+b # Punto lejano a la izquierda
    x_arriba = (a-y_arriba)/b
    y_abajo = -1  # Punto lejano a la derecha
    x_abajo = (a-y_abajo)/b
    diagonal_extendida = Line(
        ax.coords_to_point(b, 0),
        ax.coords_to_point(0,a*b),
        color=RED)

    return diagonal, diagonal_extendida

with self.voiceover(text=_("""Así, podemos emplear la diagonal
```

```
del rectangulo de base 1 y altura 2 para multiplicar 2 por
cualquier número.""")) as tracker:

    a=2
    b=3
    rectángulo = Rectangle(height=1.0*a,
                           width=1.0).next_to(ax.coords_to_point(0, 0),
                                              UR,
                                              buff=0)
    diagonal, diagonal_extendida = multiplicaEjeVertical(a, b, ax)

    self.play(FadeIn(rectángulo),
              Create(diagonal),
              run_time=tracker.duration+0.2)
    self.play(FadeOut(rectángulo))

with self.voiceover(text=_("""Para multiplicar por 3 basta
dibujar una paralela que interseque el eje horizontal en 3. El
valor del producto corresponde al punto de corte con el eje
vertical.""")) as tracker:

    xlabel,ylabel=coordenadas_xy(b, a*b, ax)

    self.play(Indicate(xlabel,
                       scale_factor=2.5),
              ReplacementTransform(diagonal.copy(),
                                   diagonal_extendida))
    self.pausa(tracker.duration*2/3)
    self.play(Indicate(ylabel,
                       scale_factor=2.5))
    self.pausa()
    self.play(*[FadeOut(mob)for mob in self.mobjects]) # borrado de todo


#from manim import *
#import numpy as np

#class RectasConProporciones(Scene):
def line_intersection(l1, l2):
    """
    Devuelve el punto de intersección entre dos rectas dadas
    por pares de puntos (p1, p2), (p3, p4).
    """
    p1, p2 = l1
    p3, p4 = l2

    x1, y1 = p1[:2]
    x2, y2 = p2[:2]
    x3, y3 = p3[:2]
    x4, y4 = p4[:2]

    denom = (x1-x2)*(y3-y4) - (y1-y2)*(x3-x4)
    px = ((x1*y2-y1*x2)*(x3-x4)-(x1-x2)*(x3*y4-y3*x4))/denom
    py = ((x1*y2-y1*x2)*(y3-y4)-(y1-y2)*(x3*y4-y3*x4))/denom

    return np.array([px, py, 0])

# Rectas blancaes (oblicuas y no paralelas)
blanca1 = Line(start=LEFT*6 + DOWN*1, end=RIGHT*6 + UP*2, color=WHITE)
blanca2 = Line(start=LEFT*6 + UP*1,   end=RIGHT*6 + DOWN*1, color=WHITE)

# Primera recta azul
azul1 = Line(start=LEFT*6 + UP*2, end=RIGHT*6 + DOWN*2, color=BLUE)

# Segunda recta azul paralela
desplazamiento = UP*0.5
azul2 = azul1.copy().shift(desplazamiento)
```

```
        # Intersecciones
        p_1 = line_intersection(
            (blanca1.get_start(), blanca1.get_end()),
            (azul1.get_start(), azul1.get_end())
        )

        p_alpha = line_intersection(
            (blanca2.get_start(), blanca2.get_end()),
            (azul1.get_start(), azul1.get_end())
        )

        p_beta = line_intersection(
            (blanca1.get_start(), blanca1.get_end()),
            (azul2.get_start(), azul2.get_end())
        )

        p_alpha_beta = line_intersection(
            (blanca2.get_start(), blanca2.get_end()),
            (azul2.get_start(), azul2.get_end())
        )

        # Marcas de puntos
        d1 = Dot(p_1)
        da = Dot(p_alpha)
        db = Dot(p_beta)
        dab = Dot(p_alpha_beta)

        # Etiquetas
        t1 = MathTex("1").next_to(d1, UP)
        ta = MathTex(r"\alpha").next_to(da, DOWN)
        tb = MathTex(r"\beta").next_to(db, UP)
        tab = MathTex(r"\alpha\beta").next_to(dab, DOWN)

        with self.voiceover(text=_("""Si lo piensa verá que ni
        siquiera necesitamos dos ejes perpendiculares para representar
        el producto. Bastan dos rectas que se corten en cero.""")) as tracker:

            self.play(Create(blanca1),
                      Create(blanca2),
                      FadeIn(d1, da),
                      Write(t1),
                      Write(ta))

            self.play(Create(azul1))

            self.play(FadeIn(db),
                      Write(tb)),

            self.play(ReplacementTransform(azul1.copy(),
                                           azul2))

            self.play(FadeIn(dab),
                      Write(tab))

            self.wait(2.5)
            self.play(Indicate(blanca1),
                      Indicate(blanca2))
            self.wait(0.5)
            self.play(*[FadeOut(mob)for mob in self.mobjects]) # borrado de todo
```

1.3. Escena 3 - Interpretación geométrica del producto de vectores de de $\mathbb{R}^2$

```
class L01_V04_E03_InterpretacionGeometricaDelProductoPorEscalaresR2(MiEscenaConVoz):
    def construct(self):
```

```
self.set_speech_service( AzureService(voice="es-ES-AlvaroNeural" ) )
#self.set_speech_service(GTTSService(lang="es", tld="com"))
myTemplate = TexTemplate()
myTemplate.add_to_preamble(r"""\usepackage{nacal} """)

# procedimientos auxiliares
def coordenadaX(x, ax: Axes):
    posición = UR if x>0 else UL
    rectángulo = Rectangle(height=unidad,
                           width=unidad*x,
                           color=GREY_B,
                           stroke_width=3).next_to(ax.coords_to_point(0, 0),
                                                   posición,
                                                   buff=0)
    return rectángulo

def coordenadaY(y, ax: Axes):
    posición = UR if y>0 else UL
    rectángulo = Rectangle(height=unidad*y,
                           width=unidad,
                           color=GREY_B,
                           stroke_width=3).next_to(ax.coords_to_point(0, 0),
                                                   posición,
                                                   buff=0)
    return rectángulo

def Arco(ax, b):
    unidad = 1
    arco = Arc(start_angle=PI/2,  # Ángulo de inicio (90 grados en radianes)
               angle=-PI/2,       # Ángulo de arco (-90 grados en radianes)
               radius=b*unidad,     # Radio del arco
               arc_center=ax.c2p(0,0),
               stroke_width=0.8)
    return arco

def multiplica(vector, c, axes: Axes, unidad):
    """
    Multiplica un vector de R2 por un escalar
    """
    def diagonales(x,y,c=1):
        diagonalx = Line(axes.coords_to_point(x*c, 0),
                         axes.coords_to_point(0, 1*c),
                         color=BLUE)
        diagonaly = Line(axes.coords_to_point(1*c, 0),
                         axes.coords_to_point(0, y*c),
                         color=BLUE)
        return diagonalx, diagonaly

    x,y,z = vector.coords
    # arcos
    arco1 = Arco(axes, unidad)
    arco2 = Arco(axes, unidad*c)
    # Diagonal rectángulos
    diagonalx1, diagonaly1 = diagonales(x,y)
    diagonalx2, diagonaly2 = diagonales(x,y,c)

    return arco1, diagonalx1, diagonaly1, arco2, diagonalx2, diagonaly2

# elementos generales

cabecera = Tex(_("Interpretación geométrica del producto de vectores en $\R[2]$"),
                   tex_template = myTemplate,
                   font_size=60).scale_to_fit_width(config.frame_width*.9).to_edge(UP).set_color(BLUE)

# plano de coordenadas
unidad = 673/1200
mínimo = -12.2   # Rango de X de -6 a 16
máximo =  12.2   # Rango de Y de -6 a 16
```

```
axes = NumberPlane(
    x_range=[mínimo+3, máximo],
    y_range=[mínimo+3, máximo],
    x_length=12,       # Longitud total en el eje X (16 - (-6) = 22)
    y_length=12,       # Longitud total en el eje Y (16 - (-6) = 22)
).add_coordinates()#.to_edge(RIGHT)

# inicio de la locución
self.creditos(7)
self.add(cabecera)
self.pausa_corta()

with self.voiceover(text=_("""Ahora ya podemos interpretar
geométricamente el reescalado de vectores""")) as tracker:

    self.pausa(tracker.duration)
    self.play(FadeIn(axes),
              FadeOut(cabecera))

######## sin componentes nulas ###

with self.voiceover(text=_("""Consideremos cualquier vector de
R2 sin componentes nulas.""")) as tracker:

    a  = nc.Vector([sp.symbols('x'), sp.symbols('y')])
    escalar = sp.symbols('\lambda')
    v  = a*escalar
    lv = MathTex(r"\lambda\," + a.latex() + " = ",  tex_template = myTemplate)
    s1 = MathTex(v.latex(),  tex_template = myTemplate)
    grp1 = VGroup(lv, s1).arrange(RIGHT).to_corner(UL)

    fondo = SurroundingRectangle(
        grp1,
        color=BLACK,
        fill_color=BLACK,
        fill_opacity=0.8,
        stroke_width=0,
        buff=0.15)

    self.add(fondo)
    self.play(FadeIn(grp1))
    self.pausa()

with self.voiceover(text=_("""Por ejemplo el 4 2""")) as tracker:

    a=4
    b=2
    Vect1 = nc.Vector([a,b])
    vect1 = VectorR2(Vect1.lista,
                     rpr='colum',
                     color=GOLD_A)
    vect1_dot = vect1.arrow(axes)
    vect1_tex = vect1.tex
    vect1_tex.next_to(vect1_dot.get_end(), UP)
    fondo1 = SurroundingRectangle(
        vect1_tex,
        color=BLACK,
        fill_color=BLACK,
        fill_opacity=0.8,
        stroke_width=0,
        buff=0)
    vect1_v_line = vect1.v_line(axes)
    vect1_h_line = vect1.h_line(axes)
    vect1_flecha = vect1.arrow(axes)

    self.play(FadeIn(vect1_dot, fondo1, vect1_tex, vect1_v_line, vect1_h_line))
    self.pausa()
```

```
    arco1, diagonalx1, diagonaly1, *__ = multiplica(vect1, 1, axes, unidad)


with self.voiceover(text=_("""Al reescalarlo, aplicamos el mismo factor a todas sus
componentes""")) as tracker:
    self.play(Indicate(grp1[1][0][1], scale_factor=2),
              Indicate(grp1[1][0][3], scale_factor=2),
              run_time=tracker.duration)
    self.pausa_muy_corta()

with self.voiceover(text=_("""Así pues, debemos multiplicar
gráficamente tanto la primera componente como la segunda""")) as tracker:

    rectánguloX = coordenadaX(a, axes)
    rectánguloY = coordenadaY(b, axes)

    self.wait(tracker.duration/2)
    self.play(FadeIn(rectánguloX),
              FadeIn(diagonalx1),
              run_time=tracker.duration/8)
    self.play(FadeIn(rectánguloY),
              FadeIn(diagonaly1),
              run_time=tracker.duration/8)

with self.voiceover(text=_("""Nótese que ahora una componente
está representada en el eje horizontal y la otra en el
vertical""")) as tracker:

    self.wait(tracker.duration/2)
    self.play(Indicate(rectánguloX),
              Indicate(diagonalx1),
              run_time=tracker.duration/8)
    self.play(Indicate(rectánguloY),
              Indicate(diagonaly1),
              run_time=tracker.duration/8)

with self.voiceover(text=_("""Aplicar el mismo
factor a ambas componentes significa que las traslaciones de
las diagonales deben situar las nuevas rectas de manera que
los extremos que ahora intersecan en el valor uno de cada eje,
intersequen en un mismo valor tras la traslación.""")) as tracker:

    self.wait(tracker.duration*3/5)
    self.play(FadeOut(rectánguloX), FadeOut(rectánguloY))

    arco1.set_stroke(width=3)
    self.play(Indicate(arco1))

def creaciónDeMúltiplos(tupla, factor, axes,
                        arco0,  vect0_dot, vect0_tex,
                        vect0_h_line, vect0_v_line,
                        diagonalx0, diagonaly0,
                        fondo0=[],
                        color=GOLD_A, rpr='colum', radio=0.05, unidad=673/1200,
                        run_time=2):

    Vect = factor*nc.Vector(tupla)
    vect = VectorR2(Vect.lista, rpr=rpr, color=color)
    vect_dot = vect.arrow(axes)
    vect_tex = vect.tex
    vect_tex.next_to(vect_dot.get_end(), UP)
    fondo = SurroundingRectangle(
        vect_tex,
        color=BLACK,
        fill_color=BLACK,
        fill_opacity=0.8,
        stroke_width=0,
        buff=0)
```

```
        vect_h_line = vect.h_line(axes)
        vect_v_line = vect.v_line(axes)
        vect_flecha = vect.arrow(axes)
        *__, arco, diagonalx, diagonaly = multiplica(VectorR2(tupla), factor, axes, unidad)

        self.play(FadeTransform(diagonalx0,
                                diagonalx),
                  FadeTransform(diagonaly0,
                                diagonaly),
                  ReplacementTransform(arco0,
                                       arco),
                  TransformFromCopy(vect0_dot,
                                    vect_dot),
                  TransformFromCopy(vect0_h_line,
                                    vect_h_line),
                  TransformFromCopy(vect0_v_line,
                                    vect_v_line),
                  FadeIn(fondo),
                  Write(vect_tex),
                  run_time=run_time)
        return arco, vect_dot, vect_tex, vect_h_line, vect_v_line, diagonalx, diagonaly, fondo

    with self.voiceover(text=_("""Así, al multiplicar el vector
    por 3, triplicamos sus componentes.""")) as tracker:

        factor = nc.fracc(3,1)
        lista = creaciónDeMúltiplos([a,b], factor, axes,
                                    arco1, vect1_dot, vect1_tex,
                                    vect1_h_line, vect1_v_line,
                                    diagonalx1, diagonaly1,
                                    color=GOLD_A, rpr='colum', radio=0.05, unidad=unidad,
                                    run_time=tracker.duration/2)

        arco2, vect2_dot, vect2_tex, vect2_h_line, vect2_v_line, diagonalx2, diagonaly2, fondo2 = lista


    with self.voiceover(text=_("""Si lo multiplicamos por 2, duplicamos sus
    componentes.""")) as tracker:

        factor = nc.fracc(2,1)
        lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
                                    run_time=tracker.duration/2)
        arco3, vect3_dot, vect3_tex, vect3_h_line, vect3_v_line, diagonalx3, diagonaly3, fondo3 = lista

    with self.voiceover(text=_("""Si lo multiplicamos por un
    medio, reducimos sus componentes a la mitad.""")) as tracker:

        factor = nc.fracc(1,2)
        lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
                                    run_time=tracker.duration/2)
        arco4, vect4_dot, vect4_tex, vect4_h_line, vect4_v_line, diagonalx4, diagonaly4, fondo4 = lista


    with self.voiceover(text=_("""Y si lo multiplicamos por algún
    número negativo, cambiamos el signo de sus componentes;
    invirtiendo con ello el sentido de la flecha que señala el
    vector.""")) as tracker:

        factor = -1
        lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
                                    run_time=tracker.duration/4)
        arco5, vect5_dot, vect5_tex, vect5_h_line, vect5_v_line, diagonalx5, diagonaly5, fondo5 = lista

        factor = -2
        lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
                                    run_time=tracker.duration/4)
        arco6, vect6_dot, vect6_tex, vect6_h_line, vect6_v_line, diagonalx6, diagonaly6, fondo6 = lista
```

```
with self.voiceover(text=_("""Además, cualquier vector
multiplicado por cero es un vector nulo.""")) as tracker:
    factor = 0
    lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
                                run_time=tracker.duration/2)
    arco0, vect0_dot, vect0_tex, vect0_h_line, vect0_v_line, diagonalx0, diagonaly0, fondo0 = lista

with self.voiceover(text=_("""Fíjese que la proporción entre
ambas componentes se mantiene, pues ambas componentes se
multiplican por el mismo número.""")) as tracker:


    source0 = MathTex(r"\frac{y}{x}=\alpha",
                      tex_template = myTemplate)
    añadido = MathTex(r"\frac{\lambda y}{\lambda x}=",
                      tex_template = myTemplate)
    VG0 = VGroup(añadido,source0).arrange(RIGHT)

    target0 = MathTex(r"y=\alpha \cdot x",
                      tex_template = myTemplate)

    VG0.add(target0).next_to(grp1,
                             DOWN,
                             buff=1).scale(1.5).shift(RIGHT/3)

    fondoVG0 = SurroundingRectangle(VG0,
                                    color=BLACK,
                                    fill_color=BLACK,
                                    fill_opacity=0.8,
                                    stroke_width=0,
                                    buff=0.15)

    self.add(fondoVG0)
    self.play(FadeIn(source0),
              FadeIn(añadido),
              run_time=tracker.duration*2/5)

    self.pausa(tracker.duration/6)
    self.play(Flash(grp1[1][0][1]),
              Flash(grp1[1][0][3]),
              run_time=tracker.duration/3)

with self.voiceover(text=_("""En el ejemplo, las segundas
componentes son la mitad de las primeras.""")) as tracker:

    self.play(FadeOut(añadido),
              run_time=tracker.duration/4)

with self.voiceover(text=_("""Los puntos que comparten
idéntica proporción entre sus coordenadas, constituyen una
recta que pasa por el origen.""")) as tracker:

    transform_index0 = [[0,1,2,3,4],
                        [0,3,4,1,2]]
    self.play(
        *[
            ReplacementTransform(source0[0][i],target0[0][j], rate_func=rate_functions.smooth)
            for i,j in zip(*transform_index0)
        ],
        run_time=tracker.duration/2)

    self.play(FadeOut(vect1_v_line,
                      vect2_v_line,
                      vect3_v_line,
                      vect4_v_line,
                      vect5_v_line,
                      vect6_v_line,
```

```
                    vect1_h_line,
                    vect2_h_line,
                    vect3_h_line,
                    vect4_h_line,
                    vect5_h_line,
                    vect6_h_line,
                    fondo0,
                    fondo1,
                    fondo2,
                    fondo3,
                    fondo4,
                    fondo5,
                    fondo6,
                    vect0_tex,
                    vect1_tex,
                    vect2_tex,
                    vect3_tex,
                    vect4_tex,
                    vect5_tex,
                    vect6_tex),
              run_time=tracker.duration*1/5)

    p1 = (16, 8)
    p2 = (-12, -6)
    # Conversión a puntos del plano con los ejes de coordenadas
    P1 = axes.coords_to_point(*p1)
    P2 = axes.coords_to_point(*p2)

    recta = Line(P1, P2, color=BLUE_B)

    self.play(FadeOut(vect1_dot,
                    vect2_dot,
                    vect3_dot,
                    vect4_dot,
                    vect5_dot,
                    vect6_dot),
              Indicate(recta, scale_factor=5),
              Circumscribe(target0))

    self.pausa()

    self.play(FadeOut(recta),
              FadeOut(fondo),
              FadeOut(vect0_dot),
              FadeOut(fondoVG0),
              FadeOut(grp1),
              FadeOut(source0))

with self.voiceover(text=_("""Ahora cambiemos el ejemplo a uno
con una sola componente no nula. Por ejemplo la segunda""")) as tracker:

    a  = nc.Vector([sp.symbols('x'),0])
    v  = a*escalar
    lv = MathTex(r"\lambda\," + a.latex() + " = ",  tex_template = myTemplate)
    s1 = MathTex(v.latex(),  tex_template = myTemplate)
    s2 = MathTex(r";\qquad",  tex_template = myTemplate)
    s3 = MathTex(r"x\ne0",  tex_template = myTemplate)
    grpX = VGroup(lv, s1, s2, s3).arrange(RIGHT).to_corner(UL)

    fondo = SurroundingRectangle(
        grpX,
        color=BLACK,
        fill_color=BLACK,
        fill_opacity=0.8,
        stroke_width=0,
        buff=0.15)

    a=4
```

```
        b=0
        VectX = nc.Vector([a,b])
        vectX = VectorR2(VectX.lista,
                         rpr='colum',
                         color=GOLD_A)
        #vectX_dot = vectX.dot(axes, radio=0.05)
        vectX_dot = vectX.arrow(axes)
        vectX_tex = vectX.tex
        vectX_tex.next_to(vectX_dot.get_end(), UP)
        fondoX = SurroundingRectangle(
            vectX_tex,
            color=BLACK,
            fill_color=BLACK,
            fill_opacity=0.8,
            stroke_width=0,
            buff=0)
        vectX_v_line = vectX.v_line(axes)
        vectX_h_line = vectX.h_line(axes)
        vectX_flecha = vectX.arrow(axes)

        self.play(FadeIn(fondo),
                  FadeIn(grpX),
                  run_time=tracker.duration/4)

        self.play(FadeIn(fondoX),
                  FadeIn(vect1_dot),
                  Write(vect1_tex),
                  Create(diagonalx1),
                  Create(diagonaly1),
                  run_time=tracker.duration/5)

        arco1, diagonalxX, diagonalyX, *__ = multiplica(vectX, 1, axes, unidad)

        self.play(Indicate(grpX[1][0][3],
                           scale_factor=2.5),
                  FadeTransform(vect1_dot,
                                vectX_dot),
                  FadeTransform(vect1_tex,
                                vectX_tex),
                  FadeTransform(diagonalx1,
                                diagonalxX),
                  FadeTransform(diagonaly1,
                                diagonalyX),
                  run_time=tracker.duration/2)
        self.pausa()

    with self.voiceover(text=_("""Al reescalarlo, aplicamos el
    mismo factor a todas sus componentes, pero ahora la segunda
    componente no cambia, pues siempre es cero.""")) as tracker:

        factor = nc.fracc(3,1)
        lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
                                    run_time=tracker.duration/7)
        arco2, vect2_dot, vect2_tex, vect2_h_line, vect2_v_line, diagonalx2, diagonaly2, fondo2 = lista


        factor = 2
        lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
                                    run_time=tracker.duration/7)
        arco3, vect3_dot, vect3_tex, vect3_h_line, vect3_v_line, diagonalx3, diagonaly3, fondo3 = lista

        factor = nc.fracc(1,2)
        lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
                                    run_time=tracker.duration/7)
        arco4, vect4_dot, vect4_tex, vect4_h_line, vect4_v_line, diagonalx4, diagonaly4, fondo4 = lista

        factor = -1
        lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
```

```
                                run_time=tracker.duration/7)
    arco5, vect5_dot, vect5_tex, vect5_h_line, vect5_v_line, diagonalx5, diagonaly5, fondo5 = lista

    factor = -2
    lista = creaciónDeMúltiplos([a,b], factor, axes, *lista,
                                run_time=tracker.duration/7)
    arco6, vect6_dot, vect6_tex, vect6_h_line, vect6_v_line, diagonalx6, diagonaly6, fondo6 = lista
    self.pausa_corta()

with self.voiceover(text=_("""El conjunto de sus múltiplos
constituye la recta del eje horizontal.""")) as tracker:

    punto_Ax = axes.coords_to_point(-9.2, 0)
    punto_Bx = axes. coords_to_point(12.2, 0)
    eje_x = Line(punto_Ax, punto_Bx, color=RED_B)

    self.remove(*[vectX_dot,
                  vect2_dot,
                  vect3_dot,
                  vect4_dot,
                  vect5_dot,
                  vect6_dot,
                  vectX_tex,
                  vect2_tex,
                  vect3_tex,
                  vect4_tex,
                  vect5_tex,
                  vect6_tex,
                  arco6,
                  fondoX,
                  fondo0,
                  fondo1,
                  fondo2,
                  fondo3,
                  fondo4,
                  fondo5,
                  fondo6,
                  diagonalxX,
                  diagonalx6,
                  diagonaly6])
    self.play(Indicate(eje_x),
              run_time=tracker.duration)

    self.play(FadeOut(eje_x))
    self.remove(*[grpX,fondo])

with self.voiceover(text=_("""El caso de los vectores no nulos
cuya primera componente es cero, es similar""")) as tracker:

    a  = nc.Vector([0, sp.symbols('y')])
    v  = a*escalar
    lv = MathTex(r"\lambda\," + a.latex() + " = ",  tex_template = myTemplate)
    s1 = MathTex(v.latex(),  tex_template = myTemplate)
    s2 = MathTex(r";\qquad",  tex_template = myTemplate)
    s3 = MathTex(r"y\ne0",  tex_template = myTemplate)
    grpY = VGroup(lv, s1, s2, s3).arrange(RIGHT).to_corner(UL)

    fondo = SurroundingRectangle(
        grpY,
        color=BLACK,
        fill_color=BLACK,
        fill_opacity=0.8,
        stroke_width=0,
        buff=0.15)

    self.play(FadeIn(fondo),
              FadeIn(grpY),
              run_time=tracker.duration/4)
```

```
with self.voiceover(text=_("""Sus múltiplos forman la recta
del eje vertical.""")) as tracker:

    punto_Ay = axes.coords_to_point(0,- 6)
    punto_By = axes. coords_to_point(0, 9)
    eje_y = Line(punto_Ay, punto_By, color=RED_B)

    self.play(Indicate(eje_y),
              run_time=tracker.duration)

    self.play(FadeOut(eje_y))

resumen1 = Tex(_(r"""El conjunto de múltiplos de un vector \underline{no nulo} de \R[2]"""),
               tex_template = myTemplate)
resumen2 = Tex(_(r"""se corresponde con una recta que pasa por el origen."""),
               tex_template = myTemplate).next_to(resumen1, DOWN)

with self.voiceover(text=_("""Por tanto, el conjunto de
múltiplos de cualquier vector no nulo de R 2 siempre corresponde a
una recta que pasa por el origen.""")) as tracker:

    self.play(*[FadeOut(mob)for mob in self.mobjects] )
    self.play(Write(resumen1))
    self.play(Write(resumen2))
    self.pausa(1.5)
    self.play(*[FadeOut(mob)for mob in self.mobjects]) # borrado de todo

with self.voiceover(text=_("""En cuanto al vector nulo, dado
que todo múltiplo de cero es cero, el conjunto de sus
múltiplos solo contiene el punto cero-cero.""")) as tracker:

    self.play(FadeIn(axes))

    Vect0 = nc.Vector([0,0])
    vect0 = VectorR2(Vect0.lista, rpr='colum', color=YELLOW)
    vect0_dot = vect0.dot(axes, radio=0.1)

    escalar = sp.symbols('\lambda')
    lv0 = MathTex(r"\lambda\," + Vect0.latex() + " = ",  tex_template = myTemplate)
    w0  = escalar*Vect0
    s0 = MathTex(w0.latex(),  tex_template = myTemplate)
    paralambda = MathTex(r"\text{para cualquier } \lambda",  tex_template = myTemplate)
    grp0 = VGroup(lv0, s0, paralambda).arrange(RIGHT).to_corner(UL)

    fondo0 = SurroundingRectangle(
        grp0,
        color=BLACK,
        fill_color=BLACK,
        fill_opacity=0.8,
        stroke_width=0,
        buff=0.15)

    self.add(fondo0)
    self.play(FadeIn(grp0))
    self.play(Indicate(vect0_dot),
              #FadeIn(vect0_tex),
              run_time=tracker.duration/3)
    self.wait(tracker.duration/3)
    self.play(Indicate(vect0_dot,
                       scale_factor=2.5))
    self.play(FadeOut(vect0_dot),
              #FadeOut(vect0_tex),
              FadeOut(fondo0),
              FadeOut(grp0))
```

1.4. Escena 4 - Interpretación geométrica del producto de tres números (3D)

1.4.1. Escena 3D

```
# from manim import *
# import numpy as np

class L01_V04_E04_InterpretacionGeometricaDelProductoPorEscalaresR3_3D(ThreeDScene):
    def construct(self):
        mínimo = -10
        máximo = 12

        # Crear ejes 3D
        axes = ThreeDAxes(
            x_range=[mínimo+3, máximo],
            y_range=[mínimo+3, máximo],
            z_range=[mínimo+3, máximo],
            x_length=12,
            y_length=12,
            z_length=12,
        ).add_coordinates()

        def Arcos(axes, radio):
            # Obtener la posición del punto (0,0,0) en los ejes
            origen_ejes = axes.c2p(0, 0, 0)
            # Radio en coordenadas de los ejes
            radio_coordenadas = radio
            # Convertir el radio a unidades de Manim
            radio_manim = np.linalg.norm(
                axes.c2p(radio_coordenadas, 0, 0) - axes.c2p(0, 0, 0))
            # si radio negativo nos vamos al cuadrante negativo con start_angle=PI
            signo = 0 if radio>0 else PI
            # Arco entre X e Y (en el plano XY)
            arco_xy = Arc(
                radius=radio_manim,
                start_angle=signo,
                angle=PI/2,
                arc_center=origen_ejes,
                stroke_width=1.8)
            # Arco entre Y y Z (en el plano YZ)
            arco_yz = Arc(
                radius=radio_manim,
                start_angle=signo,
                angle=PI/2,
                arc_center=origen_ejes,
                stroke_width=1.8).rotate(-PI/2, axis=UP, about_point=origen_ejes)
            # Arco entre Z y X (en el plano ZX)
            arco_zx = Arc(
                radius=radio_manim,
                start_angle=signo,
                angle=PI/2,
                arc_center=origen_ejes,
                stroke_width=1.8).rotate(PI/2, axis=RIGHT, about_point=origen_ejes)

            arcos = VGroup(arco_xy, arco_yz, arco_zx)
            return arcos

        def coordenadasYdiagonales(tupla, axes, factor=1):

            coordenadas = tupla
            Vector = nc.Vector(coordenadas)

            Vx = nc.Vector([coordenadas[0],0,0,])
```

```
    Vy = nc.Vector([0,coordenadas[1],0,])
    Vz = nc.Vector([0,0,coordenadas[2],])
    Vxy = nc.Vector([coordenadas[0],coordenadas[1],0,])

    # líneas de las coordenadas
    line_x = Line(start=np.array(axes.c2p(*Vx.lista)),
                  end=np.array(axes.c2p(*Vxy.lista)),
                  stroke_width=1.0)
    line_y = Line(start=np.array(axes.c2p(*Vy.lista)),
                  end=np.array(axes.c2p(*Vxy.lista)),
                  stroke_width=1.0)
    line_xy = Line(start=np.array(axes.c2p(*Vxy.lista)),
                   end=np.array(axes.c2p(*Vector.lista)),
                   stroke_width=1.0)
    line_z = Line(start=np.array(axes.c2p(*Vz.lista)),
                  end=np.array(axes.c2p(*Vector.lista)),
                  stroke_width=1.0)

    lineas = VGroup(line_x, line_y, line_xy, line_z)

    # Diagonales
    diagonal_x = Line3D(
        start=axes.c2p(*[factor if x != 0 else 0 for x in (0,1,0)]),
        end=axes.c2p(*Vx.lista),
        #stroke_width=1.9,
        color=RED)

    diagonal_y = Line3D(
        start=axes.c2p(*[factor if x != 0 else 0 for x in (0,0,1)]),
        end=axes.c2p(*Vy.lista),
        #stroke_width=1.9,
        color=GREEN)

    diagonal_z = Line3D(
        start=axes.c2p(*[factor if x != 0 else 0 for x in (1,0,0)]),
        end=axes.c2p(*Vz.lista),
        #stroke_width=1.9,
        color=BLUE)

    diagonales = VGroup(diagonal_x, diagonal_y, diagonal_z)

    return lineas, diagonales


def creaciónDeMúltiplos(tupla, factor, axes, arcos0, flecha0, lineas0, diagonales0,
                        color=TEAL_A, run_time=2.5):

    Vect = factor*nc.Vector(tupla)
    vect = VectorR3(Vect.lista, color=color)
    flecha = vect.arrow(axes)
    arcos = Arcos(axes, factor)
    lineas, diagonales = coordenadasYdiagonales(Vect.lista, axes, factor)


    self.play(ReplacementTransform(diagonales0,
                                   diagonales),
              ReplacementTransform(arcos0,
                                   arcos),
              ReplacementTransform(lineas0,
                                   lineas),
              TransformFromCopy(flecha0,
                                flecha),
              run_time=run_time)

    return arcos, flecha, lineas, diagonales

tupla = (4,3,2)
arcos0 = Arcos(axes, 1)
```

```
lineas0, diagonales0 = coordenadasYdiagonales(tupla, axes)

vector = VectorR3(tupla, color=TEAL_A)
flecha0 = vector.arrow(axes)


self.add(axes)
# Configurar la cámara para ver la escena en 3D
self.set_camera_orientation(phi=75 * DEGREES,
                            theta=45 * DEGREES,
                            zoom=1.1)
self.begin_ambient_camera_rotation(rate=0.2)
self.wait(2)

self.play(FadeIn(lineas0))
self.wait(1.5)
self.play(FadeIn(arcos0),
          FadeIn(diagonales0))
self.wait(7)
self.play(Create(flecha0))
self.wait(1)

factor = 2
arcos1, flecha1, lineas1, diagonales1 = creaciónDeMúltiplos(tupla, factor, axes,
                                                            arcos0, flecha0, lineas0,
                                                            diagonales0, run_time=6.5)

factor = -1
arcos2, flecha2, lineas2, diagonales2 = creaciónDeMúltiplos(tupla, factor, axes,
                                                            arcos1, flecha1, lineas1,
                                                            diagonales1)
self.wait(8.5)

self.play(FadeOut(arcos2,
                  flecha0,
                  flecha1,
                  flecha2,
                  lineas2,
                  diagonales2),
          run_time=0.5)
self.wait(0.5)

tupla = (4,0,2)
arcos0 = Arcos(axes, 1)
lineas0, diagonales0 = coordenadasYdiagonales(tupla, axes)

vector = VectorR3(tupla, color=TEAL_A)
flecha0 = vector.arrow(axes)

self.play(FadeIn(lineas0))
self.wait(1.5)
self.play(Create(flecha0))
self.wait(1.5)
self.play(FadeIn(arcos0),
          FadeIn(diagonales0))

factor = 2.5
arcos1, flecha1, lineas1, diagonales1 = creaciónDeMúltiplos(tupla, factor, axes,
                                                            arcos0, flecha0, lineas0,
                                                            diagonales0, run_time=1.5)

factor = -1.5
arcos2, flecha2, lineas2, diagonales2 = creaciónDeMúltiplos(tupla, factor, axes,
                                                            arcos1, flecha1, lineas1,
                                                            diagonales1, run_time=1.5)

self.wait(2)
```

```
        self.play(FadeOut(arcos2,
                          flecha0,
                          flecha1,
                          flecha2,
                          lineas2,
                          diagonales2),
                  run_time=0.5)
        self.wait(1)

        tupla = (0,3,0)
        arcos0 = Arcos(axes, 1)
        lineas0, diagonales0 = coordenadasYdiagonales(tupla, axes)

        vector = VectorR3(tupla, color=TEAL_A)
        flecha0 = vector.arrow(axes)


        self.play(FadeIn(lineas0),
                  Create(flecha0),
                  FadeIn(arcos0),
                  FadeIn(diagonales0))
        self.wait(1)

        factor = 4
        arcos1, flecha1, lineas1, diagonales1 = creaciónDeMúltiplos(tupla, factor, axes,
                                                                    arcos0, flecha0, lineas0,
                                                                    diagonales0, run_time=4)

        factor = -3
        arcos2, flecha2, lineas2, diagonales2 = creaciónDeMúltiplos(tupla, factor, axes,
                                                                    arcos1, flecha1, lineas1,
                                                                    diagonales1)

        self.play(FadeOut(arcos2,
                          flecha0,
                          flecha1,
                          flecha2,
                          lineas2,
                          diagonales2),
                  run_time=0.5)

        tupla = (0,0,0)
        vector = VectorR3(tupla,
                          color=YELLOW_A)
        vector_dot = vector.dot(axes,
                                radio=0.05)
        self.play(FadeIn(vector_dot))
        self.wait(2)
        self.play(Indicate(vector_dot,
                           scale_factor=2.5),
                  run_time=1.5)
        self.wait(4)

        self.play(*[FadeOut(mob)for mob in self.mobjects] )  # borrado de todo
```

1.4.2. Escena + Narración

```
<<Definición de VideoStatus y VideoMobject>>

class L01_V04_E04_InterpretacionGeometricaDelProductoPorEscalaresR3(MiEscenaConVoz):
    def construct(self):
        self.set_speech_service( AzureService(voice="es-ES-AlvaroNeural" ) )
        #self.set_speech_service(GTTSService(lang="es", tld="com"))
        myTemplate = TexTemplate()
```

```
myTemplate.add_to_preamble(r"""\usepackage{nacal} """)

self.creditos(2)

cabecera = Tex(_("Interpretación geométrica del producto por escalares en ") + r"$\R[3]$",
               tex_template = myTemplate,
               font_size=60).scale_to_fit_width(config.frame_width*.9).to_edge(UP).set_color(BLUE)

self.add(cabecera)
self.pausa_corta()

video1 = VideoMobject(
    #filename=r"./media/videos/L01_V04_ProductoDeUnVectorPorUnEscalar/480p15/Escena3D/L01_V04_E04_InterpretacionGeometric
    filename=r"./media/videos/L01_V04_ProductoDeUnVectorPorUnEscalar/1080p60/Escena3D/L01_V04_E04_InterpretacionGeometri
    speed=1.0
).scale_to_fit_width(config.frame_width*.95)

with self.voiceover(text=_("""La interpretación del reescalado de vectores es similar en R3""")) as tracker:

    self.wait(tracker.duration)
    self.play(*[FadeOut(mob)for mob in self.mobjects])
    self.add(video1)

with self.voiceover(text=_("""Consideremos primero un vector de R3 sin componentes nulas.""")) as tracker:

    self.wait(max(2,tracker.duration))

with self.voiceover(text=_("""Dibujemos rectas que corten los ejes correspondientes a cada componente según su valor, y

    self.wait(max(3,tracker.duration))

with self.voiceover(text=_("""Y señalemos el vector con una flecha.""")) as tracker:

    self.wait(tracker.duration)

with self.voiceover(text=_("""Si multiplicamos las tres coordenadas por un mismo valor alfa, reescalamos el vector en la

    self.wait(tracker.duration)

with self.voiceover(text=_("""Cuando alfa es negativo se invierte el sentido de la flecha.""")) as tracker:

    self.wait(tracker.duration)

with self.voiceover(text=_("""Pero todos los múltiplos del vector están alineados en una recta que pasa por el origen de

    self.wait(tracker.duration)

with self.voiceover(text=_("""Cuando una de las componentes es cero, los múltiplos del vector están contenidos en el pla

    self.wait(tracker.duration)

with self.voiceover(text=_("""Si únicamente hay una componente distinta de cero, los múltiplos del vector forman la rect

    self.wait(tracker.duration+1)

with self.voiceover(text=_("""El único caso en el que los múltiplos no forman una recta, ocurre cuando el vector es nulo

    self.wait(tracker.duration)
    self.play(*[FadeOut(mob)for mob in self.mobjects]) # borrado de todo

    self.add(cabecera)

with self.voiceover(text=_("""Como en R2, el conjunto de múltiplos de cualquier vector no nulo de R3 también se correspo

    resumen1 = Tex(_(r"""El conjunto de múltiplos de un vector \underline{no nulo} de \R[3]"""),
                   tex_template = myTemplate)
    resumen2 = Tex(_(r"""se corresponde con una recta que pasa por el origen."""),
                   tex_template = myTemplate).next_to(resumen1, DOWN)
```

```
            self.play(Write(resumen1))
            self.play(Write(resumen2))
            self.pausa(2)
            self.play(*[FadeOut(mob)for mob in self.mobjects]) # borrado de todo
```

1.5. Escena 5 - Resumen

```
class L01_V04_E05_Resumen(MiEscenaConVoz):
    def construct(self):
        self.set_speech_service( AzureService(voice="es-ES-AlvaroNeural" ) )
        #self.set_speech_service(GTTSService(lang="es", tld="com"))
        myTemplate = TexTemplate()
        myTemplate.add_to_preamble(r"""\usepackage{nacal} """)

        self.creditos()

        titulo = Title(_("Producto de vectores de ") + r"$\R[n]$" + _(" por escalares")  ,
                    tex_template = myTemplate,
                    font_size=65).set_color(BLUE)
        self.add(titulo)
        self.pausa()

        # Resumen
        resumen = Tex(_(r"\textbf{Lo más importante:}"),
                 tex_template = myTemplate,
                 font_size=50).set_color(ORANGE).next_to(titulo, DOWN, aligned_edge=LEFT)


        with self.voiceover(text=_("""La visualización de la interpretación geométrica del producto es muy limitada, pues no pue

            cvab = MathTex(r"\elemRp{\lambda\Vect{a}}{i}", tex_template = myTemplate)
            cvb  = MathTex(r"\lambda\eleVRpE{a}{i}", tex_template = myTemplate)
            igual = MathTex(r"=",          tex_template = myTemplate,)
            eq_suma = VGroup(cvab,igual,cvb).arrange(RIGHT).scale(1.5)

            donde = Tex(_("donde"))
            indices = MathTex(r"i=1:n", tex_template = myTemplate)
            pc_indices = VGroup(donde,indices).arrange(RIGHT, buff=1)
            grp3 = VGroup(eq_suma, pc_indices).arrange(RIGHT, buff=1)

            self.add(resumen)
            self.pausa(tracker.duration/2)
            self.play(FadeIn(grp3))

        with self.voiceover(text=_("""Las componentes...""")) as tracker:
            self.play(Indicate(cvab[0][4:6],
                               scale_factor=2.5),
                      run_time=tracker.duration)

        with self.voiceover(text=_("""de un vector reescalado por un valor lambda,""")) as tracker:
            self.play(Indicate(cvab[0][1:3],
                               scale_factor=2.5),
                      run_time=tracker.duration)

        with self.voiceover(text=_("""son las componentes del vector original...""")) as tracker:
            self.play(Indicate(cvb[0][2:5],
                               scale_factor=2.5),
                      run_time=tracker.duration)

        with self.voiceover(text=_("""multiplicadas por dicho valor lambda.""")) as tracker:
            self.play(Indicate(cvb[0][0],
```

```
                        scale_factor=2.5),
                run_time=tracker.duration)

        with self.voiceover(text=_("""Por tanto el operador selector es asociativo respecto del producto, y los paréntesis puede

            source0 = MathTex(r"\elemRp{\lambda\Vect{a}}{i}",
                              tex_template = myTemplate).next_to(grp3, DOWN, buff=1.2).scale(2)[0]
            target0 = MathTex(r"\lambda\eleVRpE{a}{i}",
                              tex_template = myTemplate).next_to(grp3, DOWN, buff=1.2).scale(2)[0]
            source1 = target0.copy()
            target1 = source0.copy()

            source2 = source0.copy()
            target2 = target0.copy()

            VG0 = VGroup(source0,target0)

            transform_index0 = [[0,1,2,3,4,5],
                                [1,0,2,5,3,4]]

            self.add(source0)

            self.wait(tracker.duration/5)

            self.play(
                *[
                    Transform(source0[i],
                              target0[j],
                              rate_func=rate_functions.there_and_back_with_pause)
                    for i,j in zip(*transform_index0)
                ],
                run_time=4*tracker.duration/5)

            self.play(*[FadeOut(mob)for mob in self.mobjects]) # borrado de todo
            self.pausa_larga()
```

2. Trozos comunes de código

2.1. Carga de la librería Manim y NacAL

```
from manim import *
from manim_voiceover import VoiceoverScene
from manim_voiceover.services.azure import AzureService
import nacal as nc
import sympy as sp
# PARA LA TRADUCCIÓN (pero no sé generar los vídeos traducidos)
from manim_voiceover.translate import get_gettext
# It is good practice to get the LOCALE and DOMAIN from environment variables
import os
LOCALE = os.getenv("LOCALE")
DOMAIN = os.getenv("DOMAIN")
# The following function uses LOCALE and DOMAIN to set the language, and
# returns a gettext function that is used to insert translations.
_ = get_gettext()
```

Definición de algunas subclases con definiciones generales

```
class MiEscenaConVoz(VoiceoverScene):
    def pausa_muy_corta(self, n=0.3):
        self.wait(n)
    def pausa_corta(self, n=0.5):
```

```
        self.wait(n)
    def pausa(self, n=1):
        self.wait(n)
    def pausa_media(self, n=1.5):
        self.wait(n)
    def pausa_larga(self, n=3):
        self.wait(n)
    def pausa_muy_larga(self, n=5):
        self.wait(n)

    <<Créditos en distintas partes de la pantalla>>
```

```
def creditos(self, variante=1):
    def analisis_opcion_elegida(tipo):
        'Análisis de las opciones de eliminación elegidas'
        lista = [100,20,10,4,2,1]
        opcion = set()
        for t in lista:
            if (tipo - (tipo % t)) in lista:
                opcion.add(tipo - (tipo % t))
                tipo = tipo % t
        return opcion
    copyright = Tex(r"\textcopyright{\;} 2026\; Marcos Bujosa  ")
    if 1 in analisis_opcion_elegida(variante):
        stampDcha  = VGroup(copyright.copy()).rotate( PI/2).scale(0.5).to_edge(RIGHT, buff=0.1).set_color(GRAY_D)
        self.add(stampDcha)
    if 2 in analisis_opcion_elegida(variante):
        stampIzda  = VGroup(copyright.copy()).rotate(-PI/2).scale(0.5).to_edge(LEFT, buff=0.1).set_color(GRAY_D)
        self.add(stampIzda)
    if 4 in analisis_opcion_elegida(variante):
        stampBottom= VGroup(copyright.copy()).rotate(    0).scale(0.5).to_edge(DOWN, buff=0.1).set_color(GRAY_D)
        self.add(stampBottom)
    if 10 in analisis_opcion_elegida(variante):
        stampTop   = VGroup(copyright.copy()).rotate(    0).scale(0.5).to_edge(  UP, buff=0.1).set_color(GRAY_D)
        self.add(stampTop)
```

```
class ZCreditos(Scene):
    def construct(self):
        copyright = Tex(r"\textcopyright{\;} 2026 \; Marcos Bujosa")
        github = Tex(r"\texttt{https://github.com/mbujosab}").next_to(copyright, DOWN)
        CGG  = VGroup(copyright,github).scale(1.1)
        self.add(CGG)
        self.wait(10)
```

Definición Vectores en R2 y R3

Estas clases definen vectores en $\mathbb{R}^2$ y $\mathbb{R}^3$ para ser representados con dibujos.

```
class VectorR2():
    def __init__(self, lista, rpr='c', color=GRAY_B):
        """Inicializa un Vector con una lista"""
        coords = lista + [0] if len(lista)<3 else lista
        self.color  = color
        #self.coords = tuple(coords)
        #self.coords = tuple([float(i) for i in coords])
        self.coords = tuple([int(i) if float(i)==int(i) else float(i) for i in coords])
        self.Vector = nc.Vector(self.coords[:2], rpr)
        self.tex    = MathTex(self.Vector.latex(), color=self.color).scale(0.8)

    def dot(self, ejes, radio=0.08):
```

```
        return Dot(ejes.c2p(*self.coords), radius=radio, color=self.color)

    def v_line(self, ejes):
        return ejes.get_vertical_line(ejes.c2p(*self.coords), color=self.color)

    def h_line(self, ejes):
        return ejes.get_horizontal_line(ejes.c2p(*self.coords), color=self.color)

    def arrow(self, ejes):
        return ejes.get_vector(self.Vector.lista, stroke_color = self.color, stroke_width=4)
```

```
class VectorR3():
    def __init__(self, lista, rpr='c', color=GRAY_B):
        """Inicializa un Vector con una lista"""
        coords = lista + [0] if len(lista)<3 else lista
        self.color  = color
        #self.coords = tuple(coords)
        self.coords = tuple([int(i) if float(i)==int(i) else float(i) for i in coords])
        self.Vector = nc.Vector(self.coords, rpr)
        self.tex    = MathTex(self.Vector.latex(), color=self.color).scale(0.8)
        self.proyXY = self.coords[:-1] + (0,)
        self.proyZ  = (0,0,) + (self.coords[-1],)

    def dot(self, ejes, radio=0.08):
        return Dot3D(ejes.c2p(*self.coords), radius=radio, color=self.color)

    def x_line(self, ejes):
        return ejes.get_vertical_line(ejes.c2p(*self.proyXY), color=self.color)

    def y_line(self, ejes):
        return ejes.get_horizontal_line(ejes.c2p(*self.proyXY), color=self.color)

    def z_line(self, ejes):
        return Line3D(start=ejes.c2p(*self.proyZ), end=ejes.c2p(*self.coords), thickness=0.01)

    def v_line(self, ejes):
        return Line3D(start=ejes.c2p(*self.proyXY), end=ejes.c2p(*self.coords), thickness=0.01)

    def xy_line(self, ejes):
        return Line3D(start=np.array([0,0,0]), end=ejes.c2p(*self.proyXY), thickness=0.01)

    def proy_x_line(self, ejes):
        return ejes.get_horizontal_line(ejes.c2p(*self.proyXY), color=self.color)

    def proy_y_line(self, ejes):
        return ejes.get_horizontal_line(ejes.c2p(*self.coords), color=self.color)

    def proy_y_line(self, ejes):
        return Line3D(start=ejes.c2p(*self.proyZ), end=ejes.c2p(*self.coords), thickness=0.01)

    def arrow(self, ejes):
        return Arrow3D(
            start=np.array(ejes.c2p(0, 0, 0)),
            end=np.array(ejes.c2p(*self.coords)),
            resolution=8,
            color = self.color )
```

Insertar un vídeo en una escena

```
import cv2 # needs opencv-python https://pypi.org/project/opencv-python/
from PIL import Image, ImageOps
from dataclasses import dataclass
```

```python
@dataclass
class VideoStatus:
    time: float = 0
    videoObject: cv2.VideoCapture = None
    def __deepcopy__(self, memo):
        return self

class VideoMobject(ImageMobject):
    '''
    Following a discussion on Discord about animated GIF images.
    Modified for videos

    Parameters
    ----------
    filename
        the filename of the video file

    imageops
        (optional) possibility to include a PIL.ImageOps operation, e.g.
        PIL.ImageOps.mirror

    speed
        (optional) speed-up/slow-down the playback

    loop
        (optional) replay the video from the start in an endless loop

    https://discord.com/channels/581738731934056449/1126245755607339250/1126245755607339250
    2023-07-06 Uwe Zimmermann & Abulafia
    2024-03-09 Uwe Zimmermann
    '''
    def __init__(self, filename=None, imageops=None, speed=1.0, loop=False, **kwargs):
        self.filename = filename
        self.imageops = imageops
        self.speed    = speed
        self.loop     = loop
        self._id = id(self)
        self.status = VideoStatus()
        self.status.videoObject = cv2.VideoCapture(filename)

        self.status.videoObject.set(cv2.CAP_PROP_POS_FRAMES, 1)
        ret, frame = self.status.videoObject.read()
        if ret:
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            img = Image.fromarray(frame)

            if imageops != None:
                img = imageops(img)
        else:
            img = Image.fromarray(np.uint8([[63, 0, 0, 0],
                                            [0, 127, 0, 0],
                                            [0, 0, 191, 0],
                                            [0, 0, 0, 255]
                                           ]))
        super().__init__(img, **kwargs)
        if ret:
            self.add_updater(self.videoUpdater)

    def videoUpdater(self, mobj, dt):
        if dt == 0:
            return
        status = self.status
        status.time += 1000*dt*mobj.speed
        self.status.videoObject.set(cv2.CAP_PROP_POS_MSEC, status.time)
        ret, frame = self.status.videoObject.read()
        if (ret == False) and self.loop:
            status.time = 0
```

```
            self.status.videoObject.set(cv2.CAP_PROP_POS_MSEC, status.time)
            ret, frame = self.status.videoObject.read()
        if ret:
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)  # needed here?
            img = Image.fromarray(frame)

            if mobj.imageops != None:
                img = mobj.imageops(img)
            mobj.pixel_array = change_to_rgba_array(
                np.asarray(img), mobj.pixel_array_dtype
            )
```

Mover miniescenas a un subdirectorio auxiliar

```
mkdir -p /home/marcos/SynologyDrive/ReposGH/Docencia/VideosMates2/lib/media/videos/L01_V04_ProductoDeUnVectorPorUnEscalar/480p15
mv $fichero aux_movie_files/
```

```
mv $fichero aux_movie_files/
```

3. Rodando: 1,2,3... ¡acción!

1. Generamos un fichero `mpeg` por cada escena

   - Versión de poca calidad:

   ```
   echo $escena | manim -pql $fichero.py --disable_caching
   ```

   - Versión con calidad HD1080:

   ```
   echo $escena | manim -qh $fichero.py --disable_caching
   ```

2. Concatenamos las escenas en un único fichero `mpeg` y añadimos música de fondo.

   - Montando la versión de baja resolución

   ```
   ln -s -f "$(pwd)/$subdir/ZCreditos/$calidad/ZCreditos.mp4" "$(pwd)/$subdir/$video/$calidad/ZCreditos.mp4"
   rm -f $subdir/$video/$calidad/$video.mp4 list.txt
   for f in $subdir/$video/$calidad/*.mp4; do echo file \'$f\' >> list.txt; done
   ffmpeg -f concat -safe 0 -i list.txt -c copy $subdir/$video/$calidad/$video.mp4
   mkdir -p tmp

   ffmpeg -i $subdir/$video/$calidad/$video.mp4 -i $music.mp3  -filter_complex "[0:a]apad[main]; [1:a]volume=0.03,apad[A]; [
   ```

   - Montando la versión de resolución HD1080

   ```
   ln -s -f "$(pwd)/$subdir/ZCreditos/$calidad/ZCreditos.mp4" "$(pwd)/$subdir/$video/$calidad/ZCreditos.mp4"
   rm -f $subdir/$video/$calidad/$video.mp4 list.txt
   for f in $subdir/$video/$calidad/*.mp4 ; do echo file \'$f\' >> list.txt; done
   ffmpeg -f concat -safe 0 -i list.txt -c copy $subdir/$video/$calidad/$video.mp4
   mkdir -p tmp
   ffmpeg -i $subdir/$video/$calidad/$video.mp4 -i $music.mp3 -filter_complex "[0:a]apad[main]; [1:a]volume=0.04,apad[A]; [m
   ```

3. Fundimos a negro los últimos segundos del vídeo (y la música).

```
dur=$(ffprobe -loglevel error -show_entries format=duration -of default=nk=1:nw=1 "tmp/$video.mp4")
offset=$(bc -l <<< "$dur"-7)
ffmpeg -i "tmp/$video.mp4" \
       -filter_complex "[0:v]fade=type=in:duration=0,fade=type=out:duration=6:start_time='$offset'[v];[0:a]afade=type=out
       -map "[v]" -map "[a]" -y $video.mp4
```

4. Traducción

   Esto no me ha funcionado.

```
xgettext -o locale/$fichero.pot $fichero.py

msginit -i locale/$fichero.pot -o locale/en/LC_MESSAGES/$fichero.po --locale=es --no-translator

manim_translate -s es -t en -d $fichero -l locale $fichero.py

manim_render_translation $fichero.py -s $escena -d $fichero -l en
```

```
manim_render_translation $fichero.py -s $escena -d $fichero -l en -ql
```